Informatik
Hauptcampus
H O C H
S C H U L E
T R I E R

Entwicklung einer mobilen Anwendung zur automatisierten Erkennung und Übersetzung japanischer Texte in Bildern

Bachelor-Teamprojekt

Leonie Kaschek
Nagina Nasir
Franziska Kaschek

Betreuerin: Prof. Dr. Claudia Schon

Trier, den 26. August 2025



Fahrplan

1. Kurzer Überblick
2. Projektorganisation
3. Entwicklungsprozess
4. Evaluation
5. Live-Demo
6. Technische Einschränkungen
7. Herausforderungen
8. Lessons Learned / Takeaways



Überblick

Stellen Sie sich vor, Sie sind in Japan ...



Beschilderung



Produktverpackungen



Literatur

Bildquelle: unsplash

Überblick

Motivation:

- Abbau von Sprachbarrieren durch das komplexe japanische Schriftsystem



Zielsetzung:

Entwicklung einer mobilen App, die

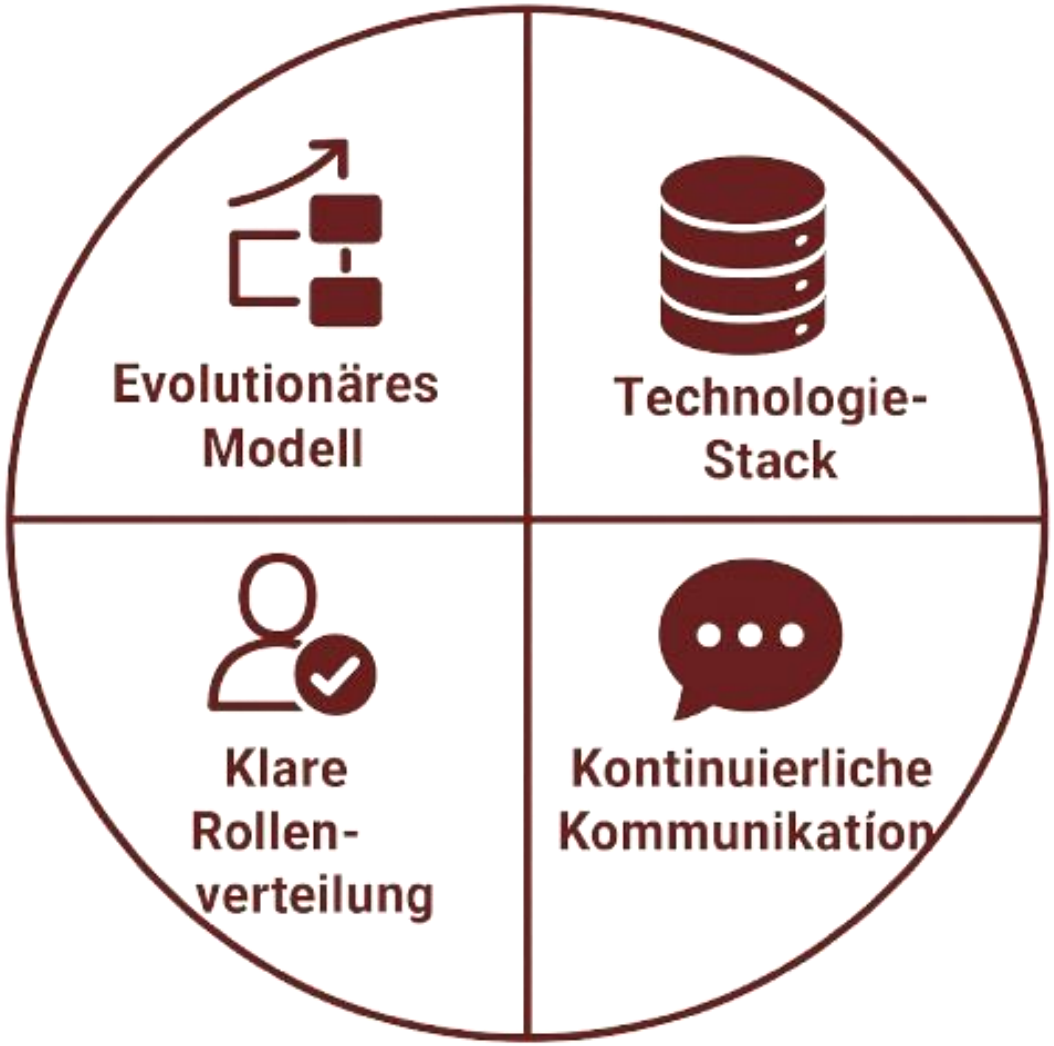
- japanische Texte in Bildern erkennt
- automatisch übersetzt
- Furigana (Lesehilfe) ergänzt

Zielgruppen:

- Japanischlernende
- Touristen



Projektorganisation

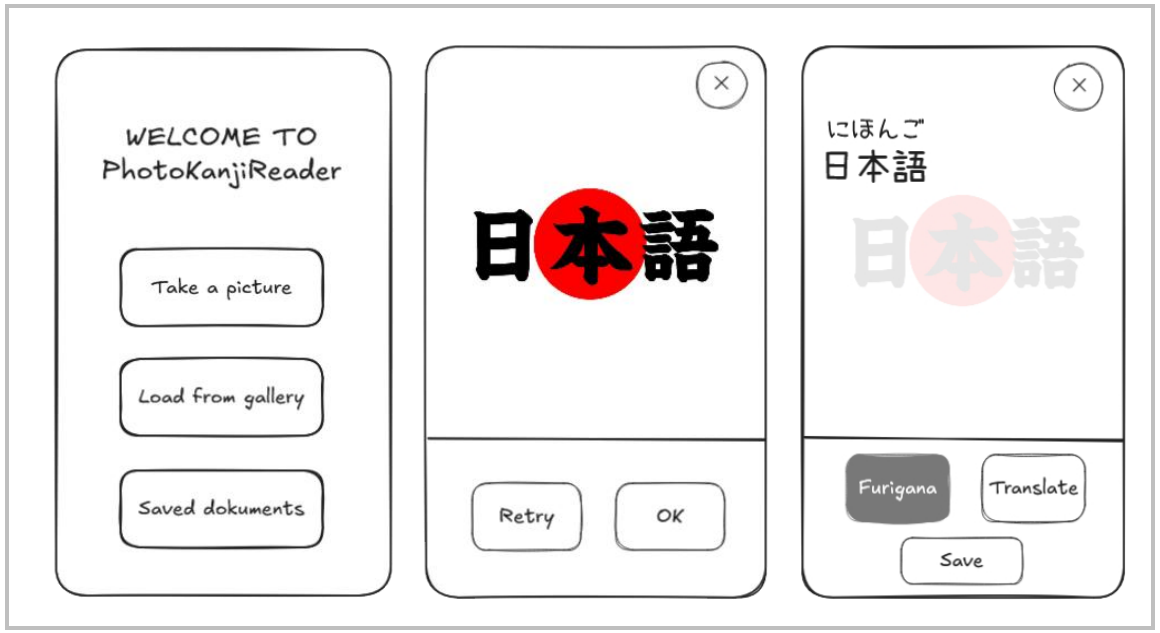


React Native
Expo
HUGGING FACE
kuromoji
ML Kit

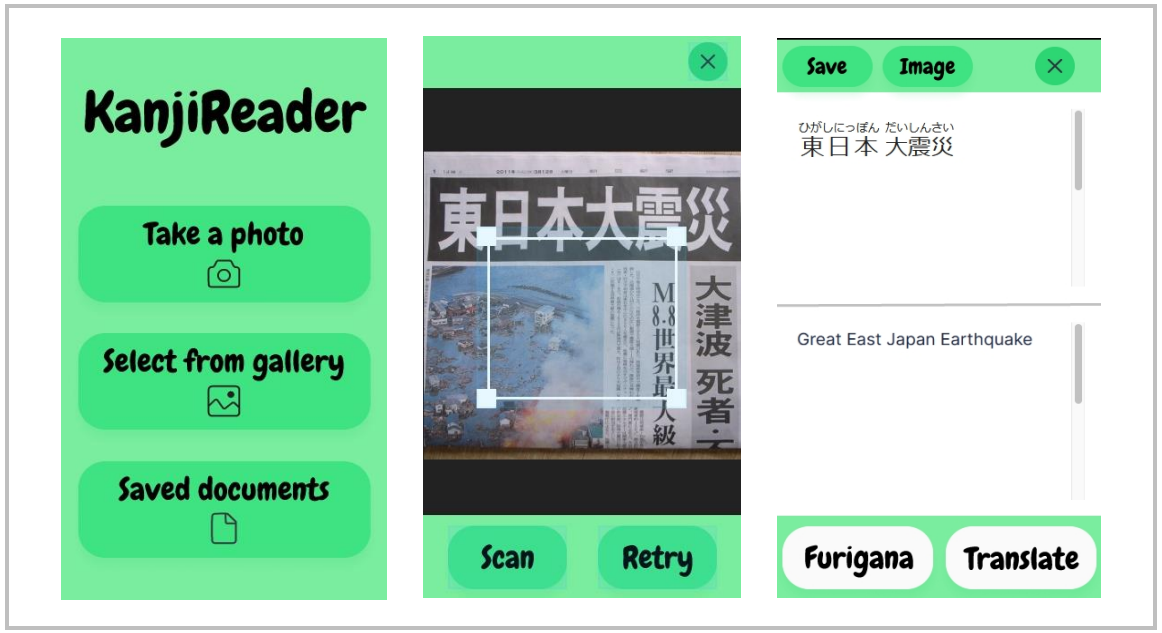
GitLab

Entwicklungsprozess

Prototypen



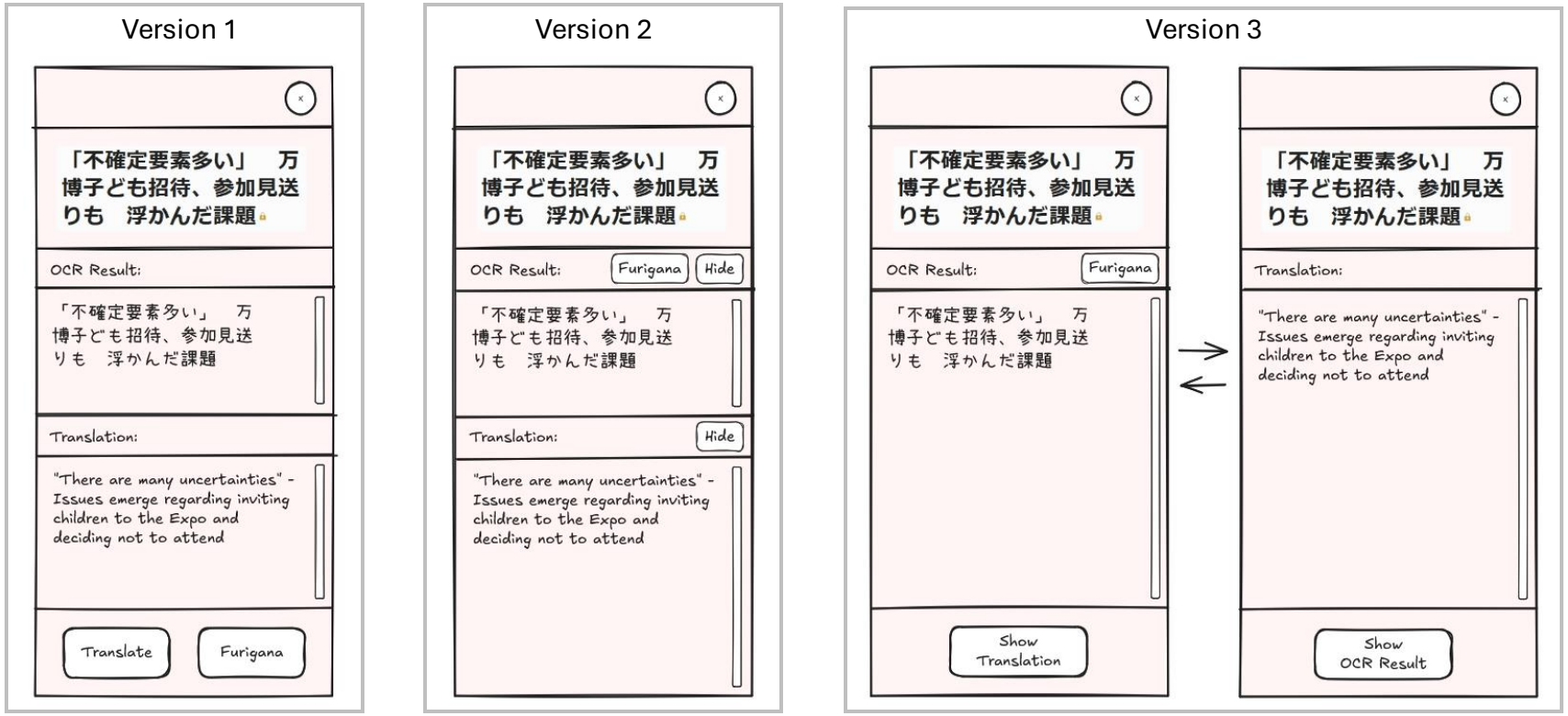
Erstes Mockup



Click-Dummy

Entwicklungsprozess

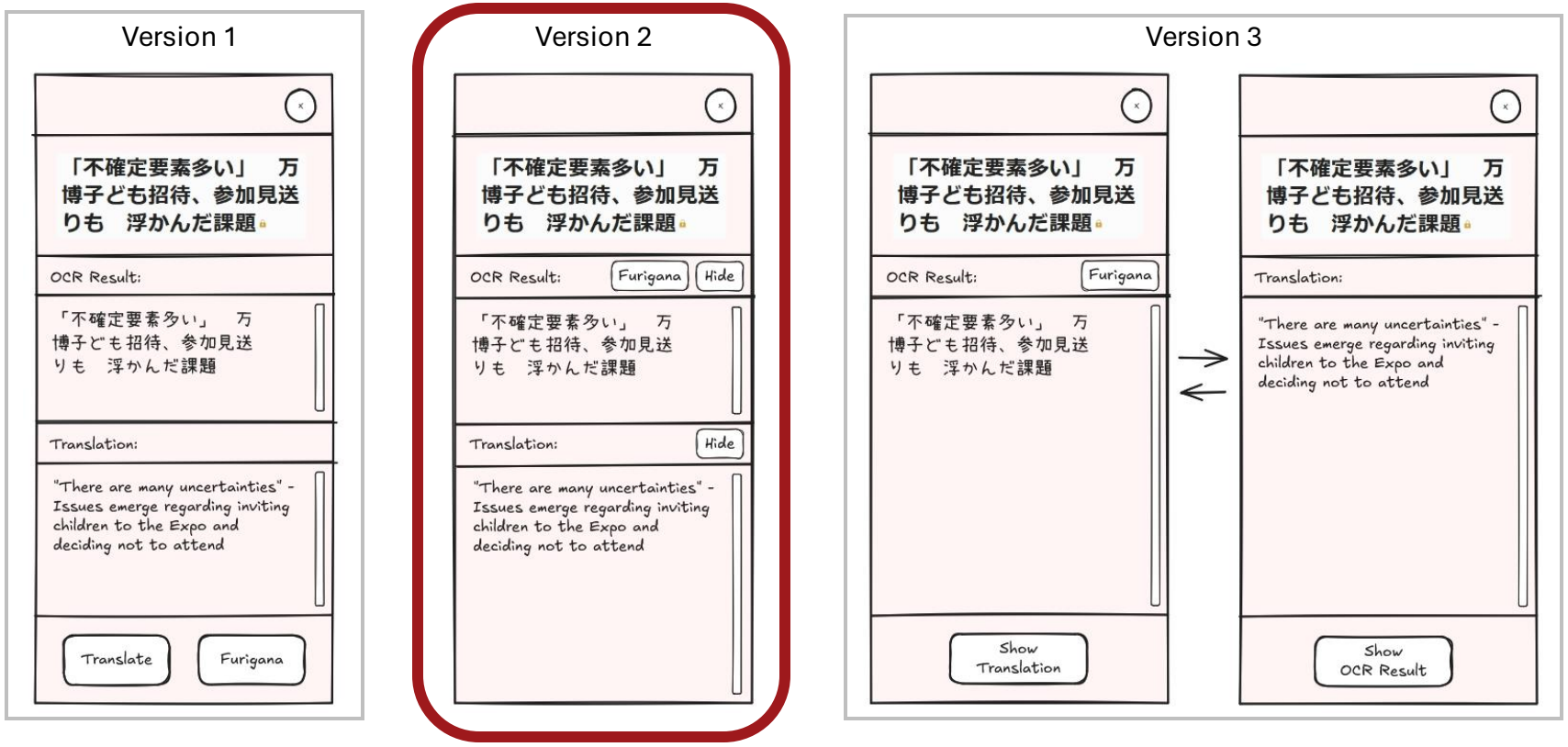
Prototypen



Drei Mockup-Varianten des Post-OCR-Screens

Entwicklungsprozess

Prototypen



Drei Mockup-Varianten des Post-OCR-Screens

Entwicklungsprozess

OCR-Komponente



1. Herausforderung:

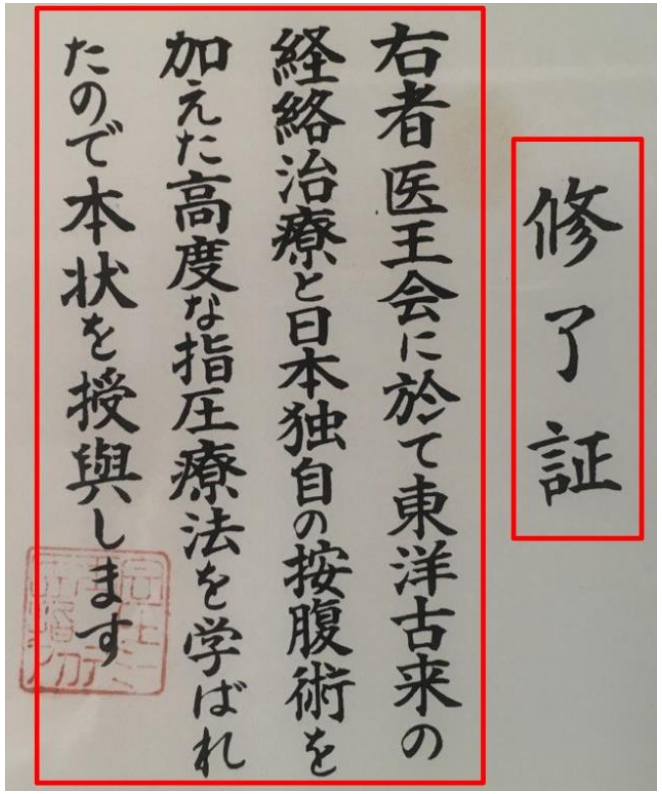
- Suche nach React-Native-kompatibler On-Device OCR-Bibliothek

Lösung:

- **1. Ansatz:** JavaScript Wrapper *Tesseract.js*
- ***2.* Ansatz:** React Native Modul *react-native-tesseract-ocr*
- **3. Ansatz:** React Native Modul: *@react-native-ml-kit/text-recognition*

Entwicklungsprozess

OCR-Komponente



2. Herausforderung:

- Vertikale Texte werden nicht in korrekter Reihenfolge erkannt

Lösung:

- Vertauschung der Textblöcke und –zeilen mithilfe der geometrischen Informationen

Entwicklungsprozess

Übersetzungskomponente

Google Cloud Translation API

- Über einfachen HTTP-Request Text übersetzen



Herausforderung:

- **API**: kostenpflichtig & durch Nutzungslimit **eingeschränkt**
- Für unsere App mit wiederholter OCR-Übersetzung **nicht nachhaltig**





Lösung:

- Umstieg auf eine **freie Alternative** → Hugging Face API

Entwicklungsprozess

Übersetzungskomponente



Herausforderung:

- Instabilität / Netzwerkfehler „**Network request failed**“
- Fehlgeschlagene Übersetzung führten zu **Wartezeit** für die Nutzer



HUGGING FACE

- **Modell:** Helsinki-NLP/opus-mt-ja-en
- **Kostenlos**, Open-Source Modell
- Text an den API-Endpoint schicken, Übersetzung als JSON zurück.



Lösung:

- **Retry-Schleife**
  → Übersetzung nach Fehlern mehrfach **automatisch** erneut anstoßen

Entwicklungsprozess

Übersetzungskomponente

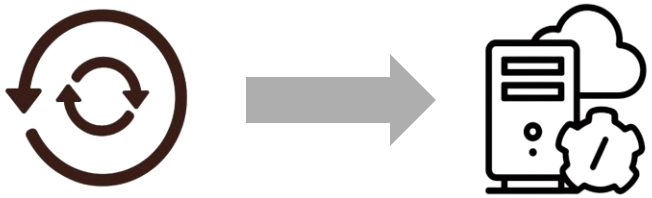
Herausforderung:

Retry-Schleife

- Ergebnis: **leichte Verbesserung**, aber Netzwerkfehler traten weiterhin auf.
- API blieb **unzuverlässig** → keine stabile Basis für die App.



HUGGING FACE



Lösung:

Selbstgehosteter Server

- Startet eine **FastAPI**-App mit Endpoint POST /translate
- Lädt Tokenizer + Modell einmalig beim Start (**vermeidet Cold-Starts** in der API)

Entwicklungsprozess

Furigana-Komponente



Herausforderung:

- Integration des Sprachanalysetools ***Kuromoji*** in React Native
- Kuromoji schwierig wegen eingebetteter **Wörterbücher**

Lösung:

1. Ansatz:
- Verwendung von *Kuromoji.js* (reine JS-Bibliothek)
- Wörterbuchdateien als Assets eingebunden

2. Ansatz:
- Integration als React-Native-Modul
- Ergebnis: vollständig lokale Verarbeitung ohne Server

Entwicklungsprozess

Furigana-Komponente

抹茶

食べる

落ち葉

立ち上がる

お菓子

お父さん

ありがとう



Herausforderung: Token-Verarbeitung und visuelle Darstellung

- Tokens enthalten Kanji + Hiragana (vor, nach, oder zwischen Kanji)
- Nur Kanji benötigen Furigana
- RN bietet keine Komponente zur korrekten Darstellung von Furigana



Lösung:

- Schrittweise Zerlegung der Token und Zuordnung der passenden Lesungen
- Eigene Komponente entwickelt, die Ruby-Annotation aus HTML simuliert

Entwicklungsprozess

Furigana-Komponente



Herausforderung: Token-Verarbeitung und visuelle Darstellung

- Tokens enthalten Kanji + Hiragana (vor, nach, oder zwischen Kanji)
- Nur Kanji benötigen Furigana
- RN bietet keine Komponente zur korrekten Darstellung von Furigana

Lösung:

- Schrittweise Zerlegung der Token und Zuordnung der passenden Lesungen
- Eigene Komponente entwickelt, die Ruby-Annotation aus HTML simuliert

Evaluation

Datensatz

ID	Description	Image	OCR ground truth	Furigana ground truth	Translation ground truth	Source
01	Mix Kanji Katakana Hiragana	image_1.jpg 庭 のバラが咲いている。	庭のバラが咲いている。	庭(にわ)のバラが咲(さ)いている。	The roses in the garden are in flower.	https://jisho.org/search/flower%20%23sentences
02	Mix Kanji Hiragana	image_2.jpg 稲 が実り、田が一面 、黄金色 になっている。	稲が実り、田が一面、黄金色 になっている。	稲(いね)が実(みの)り、田(た)が一(いち)面(めん)、黄金色(こがねいろ) になっている。	Rice is ripening and all the rice fields are turning golden.	https://jisho.org/search/all%20%23sentences
03	Hiragana only	image_3.jpg すぐ てれを しなければいけません か。	すぐそれをしなければいけませんか。	-	Do I have to do it right away?	https://tatoeba.org/de/sentences/search?query=%E3
04	Katakana only	image_4.jpg ゴール	ゴール	-	Goal	https://jisho.org/search/%E3%82%B4%E3%83%BC%E

Tools und Metriken

OCR:

- *Dinglehopper*
- CER

Übersetzung:

- *MATEO*
- BLEU
- COMET

Furigana:

- Fehleranalyse
- Zuordnung von Fehlerklassen und Fehlerursachen

Evaluation

Ergebnisse

OCR:

- **Gedruckte Texte**
  CER: 0.1548 ≈ 15 %
- **Handschriftliche Texte**
  CER: 0.3384 ≈ 34 %
- Fehler v.a. bei vertikalen und handschriftlichen Texten

Übersetzung:

- **BLEU:**
  22.3 → 16.5 (-26 %)
- **COMET:**
  79.1 → 69.4 (-12,3 %)
- Deutlicher Qualitäts-verlust durch OCR-Fehler, ansonsten robuste Leistung

Furigana:

- 47 Fehler gesamt
- ≈ 2/3 OCR-bedingt
- ≈ 1/3 andere Ursachen
- OCR-Fehler sind Hauptquelle der Probleme

Live-Demo


KanjiReader
Take a photo
Select from gallery
Saved documents


Scan
Gallery


OCR Result:
EDIT
FURI
HIDE
マギーです。お電話ありがとうございます。ただいま、電話に出られません。御用の方は発信音の後にメッセージをどうぞ。
Translation:
HIDE
I'm Maggie. Thank you for calling. I'm sorry I can't answer the phone. Please leave a message after the beep.
Save Document


DOWNLOAD
DELETE
マギーです。お電話ありがとうございます。ただいま、電話に出られません。
御用の方は発信音の後にメッセージをどうぞ。
I'm Maggie. Thank you for calling. I'm sorry I can't answer the phone. Please leave a message after the beep.
DOWNLOAD
DELETE
Delete History

Technische Einschränkungen

Einschränkungen: 

- Nur für **Android** verfügbar
- **Sprachtechnologischen Komponenten** teilweise unzuverlässig
- **Sicherheit** und **Datenschutz** nicht ausreichend berücksichtigt

Optimierungsmöglichkeiten: 

- Erweiterung auf **iOS-Geräte**
- **Leistungsfähigere Technologien** einsetzen und Pipeline verbessern
- **Serverkommunikation absichern** und datenschutzkonform gestalten

Herausforderungen

- Ressourcenmanagement
- Einarbeitung in neue Technologien
- Entwicklung eines attraktiven GUI-Designs
- Testpersonen rekrutieren
- Evaluation



Bildquelle: Freepik

Lessons Learned / Takeaways

1. **Teamfähigkeit gestärkt**
2. **Neue technische Fähigkeiten erworben**
3. **Protokollführung und Ergebnisdokumentation verbessert**
4. **Anforderungsmanagement optimiert**
5. **Bedeutung regelmäßiger kleiner Tests erkannt**
6. **Qualitätssicherung durch Evaluation**



Herzlichen Dank!